

EDUGENIE

Google Gemini Powered Learning Assistant

Team ID: 6abc95cbdfc814753aa5fe42

Team Name: EduGenie: Google Gemini Powered Learning Assistant

Project Description

EduGenie is an AI-powered learning assistant designed to help students understand academic topics, generate personalized study support, summarize learning material, create practice questions, and receive interactive explanations. The application uses Google Gemini as the generative AI layer and presents the responses through a simple, responsive learning interface.

Core modules include an AI Learning Assistant, Topic Explanation, Summary and Notes, Quiz/Practice Question Generation, Study Planner, and a learner Dashboard for tracking study activity.

Adaptation note: This document follows the structure and documentation style of the supplied FitBuddy project document, with the project-specific content adapted to EduGenie.

Technical Architecture

Frontend: React + Vite — responsive single-page learning interface.

AI Layer: Google Gemini API — explanations, summaries, quizzes and study assistance.

Routing: react-router-dom — client-side navigation.

UI: lucide-react and reusable React components.

Storage: browser localStorage for prototype preferences and learning activity.

Hosting: Vercel for public deployment.

Architecture Description

EduGenie follows a client-side single-page architecture. The React application is built with Vite and uses an AI integration layer for Gemini requests. Presentation components, learning pages, AI services, validation helpers and storage utilities are logically separated.

Conceptual entities include User, LearningPreference, Conversation, StudyTopic, Quiz, QuizQuestion, StudyPlan and Progress. A traditional ER diagram is optional for a client-side prototype.

Pre-requisites

- React and JavaScript fundamentals.
- Vite and npm familiarity.
- Client-side routing with React Router.
- API integration and JSON concepts.
- Google Gemini API concepts and secure API-key handling.
- Git/GitHub and deployment basics.

Project Workflow

Milestone 1: Planning, Technology Selection and Architecture

- Define the learning problem, target users and core features.
- Select frontend, AI, storage and hosting technologies.
- Define architecture, routes and conceptual data model.
- Set up development environment and dependencies.

Milestone 2: Core Functionalities Development

- Build the AI chat and topic-input experience.
- Integrate Gemini-powered explanation, summary and question generation.
- Implement preferences, history and progress storage.

Milestone 3: App.jsx Development

- Define routes and entry-state handling.
- Build page components and connect reusable learning components.

Milestone 4: Frontend Development

- Create design tokens and responsive layout.
- Build reusable chat, quiz, study-plan and progress components.

Milestone 5: Testing and Deployment

- Run locally and test major flows.
- Test AI states, validation, responsiveness and navigation.
- Deploy the application publicly.

Milestone 6: Conclusion

Milestone 1: Planning, Technology Selection and Architecture

This milestone defines the educational problem EduGenie addresses, identifies learners, selects technologies suitable for an AI-assisted learning application, and establishes the architecture before implementation.

Activity 1.1: Define the Problem and Target Users

Problem Statement: Students often need immediate explanations, revision support and practice material, but learning resources can be fragmented or difficult to personalize. EduGenie provides a single conversational learning assistant for topic explanations and structured study support.

Target Users:

- School and college students seeking topic explanations.
- Learners preparing for tests and examinations.
- Students who want summaries and revision notes.
- Learners who need practice questions and quick feedback.
- Users who prefer conversational, step-by-step learning support.

Selected Ideas: Gemini-powered chat, topic explanations, summaries, quiz generation, study planning, saved learning activity and progress indicators.

Future Scope: voice interaction, document-grounded answers, teacher dashboards, multilingual support, spaced repetition and richer analytics.

Activity 1.2: Technology Stack

React: component-based interface.

Vite: development and production build.

React Router: page navigation.

Google Gemini: generative learning assistance.

CSS: responsive visual system.

localStorage: prototype preferences/activity.

Vercel: public deployment.

Git/GitHub: source control.

Activity 1.3: Architecture, Routes and Data Model

Architecture: React single-page application with reusable UI components, a Gemini AI service layer, client-side state and browser storage for prototype learning data.

Routes

/login — Login / Entry

/ — Home

/dashboard — Dashboard

/learn — AI Learning

/summary — Summary

/quiz — Practice Quiz

/study-plan — Study Plan

***** — Not Found

Conceptual Data Model

User: username, display name, learning level.

LearningPreference: subject, level, preferred style, goals.

Conversation: topic, messages, timestamps.

StudyTopic: title, subject, difficulty, notes.

Quiz: topic, questions, score, date.

StudyPlan: goal, duration, sessions, topics.

Progress: topics studied, quizzes completed, recent activity.

Activity 1.4: Development Environment

The development environment can be prepared with Node.js, npm, Git and a modern code editor. The application can be initialized with Vite and the required React dependencies.

Representative structure: `edugenie/src/components`, `pages`, `services`, `data` and `utils`, together with `App.jsx`, `index.css` and `main.jsx`.

Commands: **`npm install`** · **`npm run dev`** · **`npm run build`** · **`npm run preview`**

Security note: an API key should not be exposed in client-side production code. A secure deployment should place Gemini access behind a protected backend/server-side layer.

Milestone 2: Core Functionalities Development

This milestone implements the main learning workflow: the learner asks a question or chooses a topic, EduGenie sends a structured request to the Gemini-powered service, and the result is presented as an understandable learning response.

Activity 2.1: AI Learning Experience

- Provide topic/question input with validation.
- Allow subject, level and explanation-style preferences.
- Display the assistant response in a readable conversational format.
- Provide follow-up actions such as Explain simpler, Give an example, Summarize and Create quiz.
- Show loading and error states during AI requests.
- Keep the interface responsive.

Activity 2.2: Gemini-Powered Functions

- Topic explanation: structured explanations for the selected learner level.
- Summarization: concise revision notes.
- Quiz generation: practice questions and answer checking.
- Study planning: structured study sequence.
- Interactive tutoring: follow-up questions with supported conversation context.

AI Request and Response Flow

A typical EduGenie request follows this flow:

- Learner enters a question or topic.
- Application validates input and preferences.
- AI service prepares a structured Gemini request.
- Gemini generates the learning response.
- Application validates and formats returned content.
- Response is displayed in the learning interface.
- Relevant activity can be stored for the dashboard in a prototype.

Prompt Design Considerations

- State learner level and subject when known.
- Ask for clear, step-by-step explanations.
- Request examples when appropriate.
- Use predictable formats for quiz output.
- Do not present generated information as guaranteed fact when uncertainty is possible.
- Encourage verification of important academic information with trusted sources.

Activity 2.3: Storage and Progress

- Store prototype preferences and recent activity.
- Record studied topics and quiz attempts.
- Calculate simple activity counts.
- Scope prototype data to the demo user.
- Avoid unnecessary sensitive personal data.

Milestone 3: App.jsx Development

App.jsx connects routes, shared layout and learning pages. It composes reusable components with the application services.

Activity 3.1: Routes and Entry Guard

/login → Login
/ → Home
/dashboard → Dashboard
/learn → AI Learning
/summary → Summary
/quiz → Practice Quiz
/study-plan → Study Plan
* → Not Found

A layout guard can redirect unauthenticated demo users to login. Production authentication should use a secure identity provider or backend rather than hard-coded credentials.

Activity 3.2: Page Components

Login: demo entry and validation.

Home: introduction and learning actions.

Learn: conversational Gemini learning.

Summary: revision-oriented summaries.

Quiz: practice questions and quiz activity.

Study Plan: structured study schedule.

Dashboard: recent activity and progress.

Activity 3.3: Application Data Flow

The main flow is centered on a learner request and AI response:

- User enters the application and selects a learning action.
- Inputs and preferences are validated.
- The relevant service constructs the Gemini request.
- The response is formatted for the UI.
- Learner reads, follows up, summarizes or generates a quiz.
- Supported activity is stored for the dashboard.

Example Learning Scenarios

Scenario 1: A student enters a difficult topic, selects a level and receives a step-by-step explanation with examples.

Scenario 2: A learner preparing for an examination requests a summary and then generates a practice quiz from the same topic.

Scenario 3: A learner provides topics, available days and study time, and EduGenie creates a structured study plan.

Milestone 4: Frontend Development

Milestone 4 focuses on a clean, student-friendly interface, responsive layouts and reusable components.

Activity 4.1: Design System

Primary: #073B4C for headings and navigation.

Accent: #2A9D8F for actions and interactive elements.

Surface: #E9F5F2 for cards and panels.

Background: #F8F1E3 for soft backgrounds.

Action: #EE8968 for important actions.

Text: #17343A for body text.

Muted: #658087 for secondary information.

Typography should prioritize readability for long AI-generated explanations. Headings, lists, examples and other content should be visually distinguishable.

Activity 4.2: Reusable Components

- Navbar and Footer.
- Button and form controls.
- ChatMessage / ChatBox.
- TopicCard and ProgressCard.
- QuizCard.
- StudyPlanCard.
- Loading, Error and EmptyState components.

Responsive Design and Accessibility

- Use labelled inputs and clear instructions.
- Maintain readable contrast.
- Ensure keyboard accessibility.
- Provide visible loading and error states.
- Do not rely only on colour for status or correctness.
- Keep AI responses scannable with headings, lists and spacing.

Responsible AI Considerations

- Gemini output can contain errors; important information should be verified.
- AI content should not be presented as a substitute for teachers or trusted academic resources.
- Prompts and UI should discourage unsafe or inappropriate use.
- Minimize and protect personal data.
- Secure API credentials and apply appropriate usage limits in production.

Milestone 5: Testing and Deployment

Testing covers navigation, input validation, AI request states, generated-content rendering, quiz interactions and responsive layouts.

Activity 5.1: Local Preparation

npm install

npm run dev

npm run build

npm run preview

Activity 5.2: Testing and Verification

- TC-01: Open while logged out — Login screen.
- TC-02: Valid demo account — Home/dashboard opens.
- TC-03: Empty learning prompt — Validation message.
- TC-04: Ask learning question — Loading state and AI response.
- TC-05: Request summary — Summary displayed.
- TC-06: Generate quiz — Questions displayed.
- TC-07: Submit quiz — Score/feedback updated.
- TC-08: Create study plan — Plan displayed.
- TC-09: Open dashboard — Activity/progress shown.
- TC-10: Test mobile layout — Responsive interface remains usable.

Deployment and Operational Notes

A Vite React application can be deployed to Vercel as a static frontend. If Gemini requests are made from the browser, the API credential must not be embedded in source code. Production should use a protected server-side integration.

Recommended deployment flow

- Push source code to GitHub.
- Connect the repository to the hosting platform.
- Configure secure environment variables or secrets.
- Build the Vite application.
- Deploy and verify all routes.
- Test AI requests, error handling and usage limits.

Operational Notes

- AI responses depend on the configured Gemini model and service availability.
- Network failures should show a retry message.
- Usage limits and API costs should be monitored.
- Clearing browser storage removes prototype activity history.
- Production should use real authentication and a persistent database for cross-device history.

Exploring the Website's Web Pages

Login / Entry Page

The entry screen provides a simple way to access the application. Demo credentials can be used in a prototype; production should use secure authentication.

Home Page

The landing page introduces EduGenie as a Google Gemini powered learning assistant and provides actions such as Start Learning, Generate Quiz and Create Study Plan.

AI Learning Page

The central learning screen provides topic/question input, optional learner preferences, a conversational response area and follow-up actions. The design prioritizes readable explanations and quick interaction.

Dashboard and Learning Activity

Dashboard

The dashboard acts as the learner's home base. It can show recently studied topics, quizzes completed, study-plan progress and quick actions to continue learning.

Recent Topics

Recent topic cards provide a compact history of learning activity. Selecting a topic can reopen the relevant learning workflow or start a new session.

Progress Indicators

Simple indicators can summarize study activity, such as topics studied, quizzes attempted and study-plan sessions completed. These are activity metrics rather than formal academic grades.

Summary and Notes Page

The Summary module converts a user-provided topic or learning material into concise revision notes. Output can be organized into key concepts, important points, examples and a short recap.

Suggested Workflow

- Enter or paste a topic or learning text.
- Select learner level and summary length.
- Generate the summary through the Gemini service.
- Review the generated notes.
- Use the notes as revision support and verify important details with course materials.

The module should distinguish user-provided source material from AI-generated interpretation. A future version can ground answers in uploaded textbooks or verified course documents.

Practice Quiz and Study Plan Pages

Practice Quiz

The Quiz page generates practice questions around a selected topic. After submission, the interface displays the result and can offer explanations for incorrect answers.

Study Plan

The Study Plan page collects a learning goal, topics, available time and target duration. EduGenie then produces a structured sequence of study sessions.

Example Study-Plan Fields

- Subject or examination.
- Topics to cover.
- Number of days available.
- Daily study time.
- Preferred difficulty or priority.
- Revision and practice preferences.

AI Learning Workflow: End-to-End

1. Choose learning mode — Learn, Summary, Quiz or Study Plan.
2. Enter topic/details — validate required inputs.
3. Set preferences — level, style and goal context.
4. Submit request — send structured request to Gemini.
5. Receive response — parse and render learning content.
6. Interact — follow up, retry, summarize or quiz.
7. Review — verify and use content for study.
8. Track — record supported activity.

Quality and Limitations

- Generated answers may be inaccurate or incomplete.
- Response quality depends on prompts, model and supplied context.
- The application should handle API errors and rate limits gracefully.
- Academic evaluation should not depend solely on AI-generated answers.
- Grounded content, teacher review and secure data handling can strengthen production reliability.

Conclusion

EduGenie is a Google Gemini powered learning assistant designed to make learning support more accessible through conversational explanations, summaries, practice quizzes and study planning. The project combines a responsive React interface with an AI service layer and a learner-oriented dashboard.

The project demonstrates practical use of React, Vite, client-side routing, reusable components, browser state/storage, AI API integration and responsive UI design. Its modular structure allows additional learning capabilities to be added without redesigning the complete application.

Looking Ahead

- Secure backend integration for Gemini API access.
- Real authentication and cloud database.
- Document upload and grounded question answering from course materials.
- Voice-based learning and multilingual assistance.
- Teacher/mentor dashboards and classroom integration.
- Spaced repetition, reminders, achievements and richer analytics.
- Progressive Web App support.
- Improved source citation and factual verification workflows.

Project: EduGenie: Google Gemini Powered Learning Assistant

Team ID: 6abc95cbdfc814753aa5fe42

Team Name: EduGenie: Google Gemini Powered Learning Assistant